AI KITCHEN ASSISTANT

Comprehensive Project Report

Project Name: AI Kitchen Assistant

Technology Stack: Node.js · Express.js · Supabase · Groq AI · Imagga

Type: Full-Stack Web Application with AI Integration

Date: April 2026

Version: 1.0.0

TABLE OF CONTENTS

ABSTRACT iii

LIST OF TABLES iv

LIST OF FIGURES v

LIST OF ABBREVIATIONS & SYMBOLS vi

1. INTRODUCTION 1

1.1 General 1

1.2 System Analysis 2

1.2.1 General Context 2

1.2.2 Problem Statement 3

1.2.3 Objectives 4

1.3 System Architecture and Design 5

1.4 Methodology & Technology Stack 6

2. LITERATURE REVIEW 8

2.1 General Review 8

2.2 AI and Computer Vision in Culinary Applications 9

2.3 The Role of Large Language Models in Recipe Generation 10

3. SYSTEM DESIGN 12

3.1 System Architecture Diagram 12

3.2 Database Design 13

3.3 API Design 18

3.4 Security Design 22

4. MODULE DESCRIPTIONS 24

4.1 Authentication Module 24

4.2 AI Recipe Suggestion Module 25

4.3 Recipe Management Module 27

4.4 Meal Planner Module 28

4.5 Grocery List Module 28

4.6 Smart Pantry Module 29
4.7 Social & Community Module 29
4.8 Direct Messaging Module 30
4.9 Voice & Cooking Mode Module 30
4.10 Admin Portal Module 31
4.11 Gamification Module 32
5. IMPLEMENTATION DETAILS 33
5.1 Project File Structure 33
5.2 Key Implementation Patterns 35
5.3 Deployment Configuration 36
6. TESTING AND VALIDATION 37
6.1 API Testing 37
6.2 Input Validation Testing 39
6.3 Security Testing 40
6.4 Cross-Browser & Responsive Testing 41
7. RESULTS AND DISCUSSION 42
7.1 System Performance 42
7.2 AI Output Quality 43
7.3 Feature Completion Matrix 44
8. FUTURE SCOPE 46
9. CONCLUSION 48
10. REFERENCES 49
APPENDICES 51

ABSTRACT

The AI Kitchen Assistant is an innovative, production-ready full-stack web application designed to revolutionize the culinary experience. By integrating advanced Artificial Intelligence capabilities, including Large Language Models (LLMs) and Computer Vision, the platform offers personalized recipe suggestions, smart fridge scanning, intelligent meal planning, auto-generated grocery lists, and an interactive AI chatbot.

Built on a robust technology stack featuring Node.js, Express.js, and Supabase (PostgreSQL with Row Level Security), the system ensures high performance, security, and scalability. The frontend utilizes Vanilla JavaScript paired with a modern, dynamic, and premium glassmorphism design system to deliver a visually stunning user experience. The AI functionalities are powered by Groq (utilizing Llama 3 models) for rapid, intelligent recipe generation and natural language processing, and Imagga for accurate object detection and ingredient identification from images.

The project addresses common challenges such as food waste, meal planning fatigue, and a lack of culinary inspiration, presenting a cohesive digital environment where users can manage their pantries, engage socially through a community feed, and level up their cooking skills through gamification patterns.

LIST OF TABLES

No. | Title | Description

--- | --- | ---

Table 1 | Technology Stack Summary | Overview of frontend, backend, database, and AI technologies utilized.

Table 2 | Core API Endpoints | Summary of critical RESTful API endpoints and their authentication requirements.

Table 3 | Database Schema Overview | Description of main PostgreSQL tables and their relationships.

LIST OF FIGURES

No. | Title | Description

--- | --- | ---

Figure 1 | General System Architecture | High-level diagram representing the interaction between frontend, Express backend, Supabase, and third-party AI APIs.

Figure 2 | AI Fridge Scanning Flow | The step-by-step process of converting an image into a list of ingredients and reverse-engineering a recipe.

Figure 3 | Meal Planner Logic | Flowchart outlining the automated meal plan generation over a 7-day period.

LIST OF ABBREVIATIONS & SYMBOLS

- **AI:** Artificial Intelligence
- **API:** Application Programming Interface
- **CORS:** Cross-Origin Resource Sharing
- **DB:** Database
- **JSON:** JavaScript Object Notation
- **JWT:** JSON Web Token
- **LLM:** Large Language Model
- **REST:** Representational State Transfer
- **RLS:** Row Level Security
- **SQL:** Structured Query Language
- **UUID:** Universally Unique Identifier

---

1. INTRODUCTION

1.1 GENERAL

In the modern fast-paced world, individuals frequently encounter friction when organizing their daily meals. From managing expiring pantry items to finding creative ways to cook what is available, meal preparation often becomes a chore rather than an enjoyable experience. The rise of Artificial Intelligence (AI) presents unique opportunities to alleviate these pain points. The AI Kitchen Assistant aims to be an all-encompassing digital sous-chef, bringing state-of-the-art AI right to the user's kitchen. It acts not just as a repository of recipes, but as a proactive tool that analyzes ingredients, suggests zero-waste cooking alternatives, and automates mundane tasks like drafting grocery lists.

1.2 SYSTEM ANALYSIS

1.2.1 General Context

The culinary software space is densely populated with static recipe blogs and basic meal planners. However, most existing applications operate as passive databases. Users must search for what they want and manually curate their schedules. The demand has shifted toward proactive, intelligent systems that contextually assist the user based on real-time data—such as leveraging a photo of their open refrigerator.

1.2.2 Problem Statement

1.2.2.1 General Difficulties

Households globally contribute significantly to food waste, predominantly due to forgotten ingredients expiring or a lack of knowledge on how to combine available items.

1.2.2.2 Existing System Limitations

Standard recipe apps require manual input of every available ingredient, which is tedious. Furthermore, their search algorithms rely on exact database matches rather than creative, adaptable culinary intelligence.

1.2.2.3 Proposed Solution

By adopting Computer Vision natively through the Imagga API, users can take pictures of their ingredients and let the AI instantly identify them. Incorporating Groq's high-speed inference of the Llama-3 model enables the platform to dynamically generate recipes without needing a static, pre-existing database match. The system can synthesize brand new

meals, substitute ingredients dynamically based on dietary needs, and intelligently scale servings.

1.2.3 Objectives

- **Develop a Robust Core Platform:** Construct a scalable Express backend linked to a Supabase PostgreSQL database governed by Row Level Security.
- **Integrate Computer Vision:** Enable smart "Fridge Scanning" using Imagga to accurately detect raw ingredients from user-uploaded images.
- **Implement Advanced LLM Capabilities:** Utilize Groq for high-speed AI tasks: Recipe Suggestion, Zero-Waste Ideation, Ingredient Substitution, Natural Language Chatbotting, and 7-day Meal Plan Generation.
- **Create a Premium User Interface:** Design an aesthetically pleasing, responsive frontend emphasizing modern design philosophies (e.g., glassmorphism, dynamic animations, modern typography).
- **Build Gamification and Community:** Integrate a social feed, user ranks, XP systems, and weekly cooking challenges to foster user retention and engagement.

1.3 SYSTEM ARCHITECTURE AND DESIGN

The AI Kitchen Assistant is structured around a typical 3-tier web architecture, enhanced by external microservices for AI inference.

- **Frontend (Client Tier):** Built with Vanilla JS and CSS3. Features a modular approach (e.g., `planner.js`, `chatbot.js`, `grocery.js`). The styling is highly customized using a tailored color palette and micro-animations to create a premium feel.
- **Backend (Application Tier):** A Node.js application powered by Express.js. It handles routing, middleware validation (via `express-validator`), security (via `helmet` and `cors`), rate-limiting (via `express-rate-limit`), and orchestrates requests to external AI APIs and the database.
- **Database (Data Tier):** Hosted on Supabase (PostgreSQL). Features highly normalized tables including `profiles`, `recipes`, `meal_plans`, `grocery_lists`, `pantry`, `posts`, and `audit_logs`. Security is enforced directly at the database level using Row Level Security (RLS) policies.
- **External Microservices (AI Services):**

* **Groq API:** Interfaced for lightning-fast LLM responses. Triggers the core logic in `services/ai.js` to process JSON payloads representing customized recipes.

* **Imagga API:** Receives base64 encoded images from the backend to process visual tags and return high-confidence food-related keywords.

1.4 METHODOLOGY & TECHNOLOGY STACK

The development followed an agile methodology, iteratively implementing feature modules: Authentication -> Core Recipes -> AI Integrations -> Social Features.

Technology Stack Highlights:

- **Core Logic:** JavaScript (ES6+), Node.js (v18+)
- **Server/Framework:** Express.js
- **Database:** PostgreSQL (via Supabase), enabling real-time capabilities and built-in Auth.

- **Storage:** Supabase Storage (handling avatar uploads and recipe imagery).
- **AI Integrations:** Groq Cloud (Llama 3 70B), Imagga API.
- **Security:** JWT verification, bcrypt (internal to Supabase Auth), Express rate limiters, comprehensive parameterized SQL queries.

---

2. LITERATURE REVIEW

2.1 GENERAL REVIEW

The concept of digital recipe organization has existed since the early internet days. Initially, platforms acted purely as digital cookbooks. As mobile computing advanced, these platforms integrated basic grocery list generation based on static recipe ingredients. However, a review of current literature and existing market solutions reveals a gap: the lack of adaptability. Static recipes cannot easily adjust to missing ingredients, dietary changes, or dynamic scaling without yielding compromised results.

2.2 AI AND COMPUTER VISION IN CULINARY APPLICATIONS

Recent studies in computer vision have demonstrated the efficacy of Convolutional Neural Networks (CNNs) in object detection. In the culinary space, identifying raw ingredients is notoriously difficult due to deformation, occlusion, and lighting variations inside refrigerators. Theoretical models have proposed using multi-label classification to identify multiple ingredients in a single frame. The AI Kitchen Assistant implements this literature practically by utilizing Imagga's robust tagging API, filtering for food-specific taxonomies, and cross-referencing confidence scores above 15% to build reliable input arrays.

2.3 THE ROLE OF LARGE LANGUAGE MODELS IN RECIPE GENERATION

Large Language Models, trained on vast corpora including millions of recipes and culinary blogs, have established a deep understanding of flavor profiling and ingredient pairing. Research indicates that prompt engineering can effectively restrict LLM outputs to specific JSON schemas, turning conversational agents into structured data generators.

The AI Kitchen Assistant leverages this by providing complex system prompts (e.g., requesting exactly 21 meals for a 7-day plan, distributed across breakfast, lunch, and dinner). The literature highlights that while LLMs hallucinate, grounding them with a strict provided ingredient list significantly improves output feasibility, acting as a constraint satisfaction solver in culinary space. Furthermore, the integration of context-aware chatbots in kitchens represents a modern evolution of the classic cookbook, allowing for dynamic, two-way interaction regarding technique, nutrition, and substitutions.

---

3. SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE DIAGRAM

The application follows a three-tier client-server architecture augmented by external AI microservices.

```
graph TD
    A[Browser Client - Vanilla JS] -->|HTTP/REST| B[Express.js S
erver]

    B -->|Supabase SDK| C[Supabase PostgreSQL + Auth + Storage]

    B -->|HTTPS POST| D[Groq API - Llama 3 70B]

    B -->|HTTPS POST| E[Imagga API - Computer Vision]

    B -->|Middleware Pipeline| F[Auth / RateLimit / Validate]

    F --> B
```

Request Flow:

- The browser client sends an HTTP request with a JWT Bearer token in the Authorization header.
- Express middleware validates the token via Supabase Auth (`auth.js` middleware), checks the user's role (`adminAuth.js`), enforces rate limits (`rateLimiter.js`), and validates input (`validate.js`).
- The route handler processes the request — either querying Supabase directly or forwarding to an external AI API.
- The response is returned as JSON to the client.

3.2 DATABASE DESIGN

The database consists of 14 interconnected tables hosted on Supabase PostgreSQL. All tables enforce Row Level Security (RLS).

3.2.1 Entity Relationship Summary

Table	Purpose	Key Relationships
`profiles`	User accounts extending Supabase Auth	PK: `id` (FK to `auth.users`)
`recipes`	Central recipe storage	FK: `author_id` → `profiles.id`
`meal_plans`	User meal scheduling	FK: `user_id` → `profiles.id` , `recipe_id` → `recipes.id`
`grocery_lists`	Shopping list per user	FK: `user_id` → `profiles.id`
`recipe_likes`	Many-to-many like join table	Composite PK: (`user_id` , `recipe_id`)
`audit_logs`	Admin action tracking	FK: `admin_id` → `profiles.id`

`conversations`	DM conversation envelopes	Independent PK
`conversation_members`	DM membership join table	Composite PK: (`conversation_id` , `user_id`)
`messages`	Individual chat messages	FK: `conversation_id` , `sender_id`
`posts`	Social feed entries	FK: `author_id` , `recipe_id`
`post_likes`	Social post likes	Composite PK: (`user_id` , `post_id`)
`pantry`	Smart pantry inventory	FK: `user_id` → `profiles.id`
`weekly_challenges`	Gamification challenges	Independent PK
`follows`	Social follow relationships	Composite PK: (`follower_id` , `following_id`)

3.2.2 Key Columns — Recipes Table

Column	Type	Description
`id`	UUID	Auto-generated primary key
`author_id`	UUID	Foreign key to profiles
`title`	TEXT	Recipe name (required)
`ingredients`	JSONB	Array of ingredient strings
`steps`	JSONB	Array of instruction strings
`nutrition`	JSONB	Object: `{calories, protein, carbs, fat}`
`category`	TEXT	Constrained: Breakfast, Lunch, Dinner, Snack, Dessert
`tags`	JSONB	Array of dietary tags (e.g., Vegan, Gluten-Free)
`is_ai_draft`	BOOLEAN	Flags AI-generated recipes
`views` / `likes`	INTEGER	Engagement counters
`is_featured`	BOOLEAN	Admin-toggled featured flag

3.2.3 Database Triggers and Functions

Trigger	Event	Function
`on_auth_user_created`	After INSERT on `auth.users`	`handle_new_user()` — auto-creates profile row
`on_recipe_like`	After INSERT/DELETE on `recipe_likes`	`handle_recipe_like()` — increments/decrements `recipes.likes`

`on_post_like`	After INSERT/DELETE on `post_likes`	`handle_post_like()` — updates `posts.likes_count`
`on_new_message`	After INSERT on `messages`	`update_conv_last_message()` — updates conversation preview
`handle_updated_at_*`	Before UPDATE on multiple tables	`moddatetime()` — auto-sets `updated_at`

3.2.4 Row Level Security Policies

All 14 tables have RLS enabled. Key policies include:

- **Profiles:** Publicly viewable; only owners can update their own profile.
- **Recipes:** Publicly viewable; authenticated users can create; only owners or admins can update/delete.
- **Meal Plans / Grocery Lists:** Fully private per user; service_role has full access for backend operations.
- **Audit Logs:** Only admins can view.
- **Messages:** Only conversation members can read/send; service_role has full access.

3.3 API DESIGN

The API follows RESTful conventions with consistent JSON responses.

3.3.1 Complete API Endpoint Reference

Method	Endpoint	Auth	Description
POST	`/api/auth/signup`	—	Register new user
POST	`/api/auth/login`	—	Login, returns JWT + profile
POST	`/api/auth/admin-login`	—	Admin-only login with role check
POST	`/api/auth/logout`	✅	Server-side session invalidation
GET	`/api/auth/me`	✅	Fetch current user profile
PUT	`/api/auth/profile`	✅	Update username, bio, preferences
POST	`/api/auth/profile-picture`	✅	Upload avatar via multipart form
GET	`/api/auth/search?q=`	✅	Search users by username or UUID
GET	`/api/recipes`	—	List recipes with pagination, filter, search
GET	`/api/recipes/trending`	—	Top 10 recipes by views + likes
GET	`/api/recipes/:id`	—	Single recipe detail (increments views)

POST	`/api/recipes`	✅	Upload recipe with optional image
PUT	`/api/recipes/:id`	✅	Edit recipe (owner/admin only)
DELETE	`/api/recipes/:id`	✅	Delete recipe (owner/admin only)
POST	`/api/recipes/:id/like`	✅	Toggle like/unlike
POST	`/api/recipes/:id/remix`	✅	Fork a recipe as a new copy
POST	`/api/ai/suggest`	✅	AI recipe generation from ingredients
POST	`/api/ai/meal-plan`	✅	AI 7-day meal plan (21 recipes)
POST	`/api/ai/scan-fridge`	✅	Image → ingredient detection via Imagga
POST	`/api/ai/chat`	✅	AI Chef chatbot conversation
POST	`/api/ai/substitute`	✅	AI ingredient substitution tips
POST	`/api/ai/waste-not`	✅	Zero-waste recipe ideas from pantry
POST	`/api/ai/image-to-recipe`	✅	Reverse-engineer recipe from food photo
POST	`/api/ai/smart-scale`	✅	Intelligent recipe serving scaler
GET/POST	`/api/planner`	✅	CRUD meal plans
GET/POST	`/api/grocery`	✅	CRUD grocery list
POST	`/api/grocery/generate`	✅	Auto-generate list from meal plans
GET/POST	`/api/pantry`	✅	CRUD pantry items
POST/DELETE	`/api/social/follow/:id`	✅	Follow/unfollow user
GET	`/api/social/feed`	—	Community post feed
POST	`/api/social/posts`	✅	Share a recipe post
GET	`/api/admin/users`	🛡️	List all users (paginated)
POST	`/api/admin/users`	🛡️	Create user with role assignment
DELETE	`/api/admin/users/:id`	🛡️	Delete user
GET	`/api/admin/analytics`	🛡️	Platform statistics dashboard
GET	`/api/admin/logs`	🛡️	Audit log viewer

3.3.2 Rate Limiting Configuration

Limiter	Window	Max Requests	Applied To
`aiLimiter`	15 minutes	20	All `/api/ai/*` routes

`authLimiter`	15 minutes	10	`/api/auth/login`, `/api/auth/signup`
`generalLimiter`	15 minutes	100	`/api/recipes` public routes

3.4 SECURITY DESIGN

The system implements defense-in-depth security:

- **Helmet.js:** Sets security HTTP headers (X-Content-Type-Options, X-Frame-Options, etc.).
- **CORS Whitelist:** Configurable `FRONTEND_URL` origin restriction.
- **JWT Authentication:** Supabase-issued tokens verified server-side on every protected route.
- **Input Validation:** All POST/PUT endpoints validated via `express-validator` chains.
- **Rate Limiting:** Three tiers of rate limiting to prevent abuse.
- **Row Level Security:** Database-level access control ensuring users can only access their own data.
- **Admin Role Enforcement:** Two-layer check — middleware `authenticate` + `adminOnly` guard.
- **Audit Logging:** All admin actions (user creation, deletion, recipe featuring) are logged with timestamps and admin ID.

---

4. MODULE DESCRIPTIONS

4.1 AUTHENTICATION MODULE

Files: `backend/routes/auth.js`, `backend/middleware/auth.js`, `frontend/js/auth.js`
This module handles the complete user lifecycle:

- **Signup:** Validates email format, password strength (min 8 chars, 1 uppercase, 1 digit), and username uniqueness. Creates user via Supabase Admin API. Profile row auto-created by database trigger.
- **Login:** Authenticates via Supabase `signInWithPassword()`. Returns JWT access token and user profile (including role, avatar).
- **Admin Login:** Same as login but adds a role check — returns 403 if the user's profile role is not `admin`. Logs admin login to audit table.
- **Session Management:** Token stored in `localStorage` as `ka_token`. The `apiFetch()` helper auto-attaches Bearer token and handles 401 redirects.
- **Profile Updates:** Username change with uniqueness check, bio, dietary preferences (JSONB), and custom avatar upload via Supabase Storage.

4.2 AI RECIPE SUGGESTION MODULE

Files: `backend/routes/ai.js`, `backend/services/ai.js`, `frontend/js/dashboard.js`
The core AI module provides six distinct capabilities:

- **Recipe Suggestion (`/api/ai/suggest`):** Accepts an array of ingredients, sends a structured prompt to Groq requesting 4 diverse recipes from different world

cuisines. Each recipe includes title, ingredients, steps, nutrition, and an `image_query` for dynamic photo sourcing. Results are auto-saved to the `recipes` table with `is_ai_draft = true`.

- **7-Day Meal Plan (`/api/ai/meal-plan`):** Generates exactly 21 recipe objects (3 meals × 7 days) via a single Groq API call with an 8000 max_token budget. The prompt enforces the Breakfast → Lunch → Dinner sequence for each day.
- **Fridge Scanner (`/api/ai/scan-fridge`):** Receives a base64-encoded image, forwards it to the Imagga tagging API. Filters results by confidence > 15% and a curated food keyword list (25 categories). Returns up to 15 detected ingredient names.
- **AI Chatbot (`/api/ai/chat`):** Maintains conversational context by accepting a message history array (last 10 messages). The system prompt positions the AI as "AI Chef" with expertise in recipe guidance, ingredient swaps, nutrition coaching, and meal planning.
- **Ingredient Substitution (`/api/ai/substitute`):** Given a recipe title and a specific ingredient, returns 3 practical substitutes with explanations.
- **Zero-Waste Ideas (`/api/ai/waste-not`):** Fetches the user's pantry items sorted by expiry date, sends the 10 most urgent items to Groq for creative recipe ideation.
- **Image-to-Recipe (`/api/ai/image-to-recipe`):** Combines Imagga (for visual tags) with Groq (for recipe reconstruction) to reverse-engineer a complete recipe from a food photograph.
- **Smart Scale (`/api/ai/smart-scale`):** Intelligently scales recipe servings with non-linear adjustments for spices, leavening agents, and cooking times.

4.3 RECIPE MANAGEMENT MODULE

Files: `backend/routes/recipes.js` , `frontend/js/dashboard.js`

- **Listing:** Supports pagination, category filtering, full-text search across title/tags/ingredients, and sorting by views or date.
- **Trending Algorithm:** Combines `is_featured` priority with a weighted score: `views + (likes × 3)`.
- **Upload:** Multipart form data with image upload to Supabase Storage bucket `recipe-images`. Supports JSONB fields for ingredients, steps, tags, and nutrition.
- **Remix:** Creates a fork of any recipe, linking back via `original_recipe_id` foreign key.
- **Like System:** Toggle-based like/unlike with automatic counter managed by the `on_recipe_like` database trigger.

4.4 MEAL PLANNER MODULE

Files: `backend/routes/planner.js` , `frontend/js/planner.js`
Full CRUD for meal scheduling:

- Plans are date-based entries linking a `recipe_id` to a `planned_date` and `meal_type` (Breakfast/Lunch/Dinner/Snack).
- Frontend renders an interactive calendar view.
- Authorization ensures users can only modify their own plans.

4.5 GROCERY LIST MODULE

Files: `backend/routes/grocery.js`, `frontend/js/grocery.js`

- **Manual Mode:** Users maintain a checklist of items stored as a JSONB array.
- **Auto-Generate:** The `/generate` endpoint scans all future meal plans, aggregates unique ingredients across planned recipes, and produces a deduplicated grocery list with `from_recipe` attribution.
- Uses upsert logic — creates a new list or updates existing.

4.6 SMART PANTRY MODULE

Files: `backend/routes/pantry.js`, `frontend/js/pantry.js`

- Users track pantry items with name, quantity, and expiry date.
- Items sorted by expiry date (ascending) to surface urgent items first.
- Frontend highlights items expiring within 3 days with visual alerts.
- Integrates with the zero-waste AI feature.

4.7 SOCIAL & COMMUNITY MODULE

Files: `backend/routes/social.js`, `backend/routes/users.js`, `frontend/js/social.js`

- **Follow System:** Users can follow/unfollow others. The `follows` table enforces self-follow prevention via a CHECK constraint.
- **Community Feed:** Public timeline of recipe shares with author info, like counts, and `liked_by_me` enrichment.
- **User Profiles:** Public profile pages showing bio, avatar, follower/following counts, and authored recipes.

4.8 DIRECT MESSAGING MODULE

Files: `backend/routes/chat.js`, `frontend/js/chat.js`

- **Feedback System:** Creates a private conversation between a user and the admin for feedback/support.
- **Message Storage:** Real-time-ready message table with read status tracking.
- **Admin Inbox:** Aggregated view of all user feedback threads with participant identification.

4.9 VOICE & COOKING MODE MODULE

Files: `frontend/js/voice.js`

- Utilizes the Web Speech API (`webkitSpeechRecognition`) for hands-free cooking.
- **Cooking Mode:** Full-screen overlay displaying one step at a time with voice commands: "Next", "Back", "Read ingredients", "Exit".
- **Text-to-Speech:** Uses `SpeechSynthesisUtterance` to read steps and ingredients aloud.
- **Voice Search:** Non-continuous recognition mode for searching recipes by voice.

4.10 ADMIN PORTAL MODULE

Files: `backend/routes/admin.js`, `frontend/admin.html`, `frontend/js/admin.js`

- **User Management:** List, create (with role assignment), and delete users. Self-deletion prevention.
- **Recipe Management:** List and delete any recipe. Toggle `is_featured` status.
- **Analytics Dashboard:** Aggregated stats — total users, admin count, total recipes, cumulative views/likes, meal plan count, and top 5 recipes by views.
- **Audit Logs:** Chronological log of all admin actions with admin username attribution.

4.11 GAMIFICATION MODULE

Database: `weekly_challenges` table, `profiles.xp`, `profiles.rank_points`

- **XP System:** Users earn XP for actions. The `add_user_xp()` PostgreSQL function atomically updates both XP and rank points.
- **Rank Tiers:** 🍴 Novice (0+) → 🏠 Home Cook (100+) → 🍳 Chef de Partie (500+) → 👨‍🍳 Executive Chef (1000+).
- **Progress Bar:** Frontend renders an XP progress bar showing distance to next rank.
- **Weekly Challenges:** Admin-created cooking challenges with mystery ingredients and XP rewards.

---

5. IMPLEMENTATION DETAILS

5.1 PROJECT FILE STRUCTURE

```
ai-kitchen-assistant/
├── backend/

│   ├── app.js                    # Express app setup, middlewar
e, routes

│   ├── server.js                  # Entry point (dotenv + liste
n)

│   ├── config/

│   │   └── supabase.js           # Supabase client initializati
on

│   ├── middleware/

│   │   ├── auth.js               # JWT verification via Supabas
e

│   │   ├── adminAuth.js          # Admin role guard

│   │   ├── rateLimiter.js        # Three-tier rate limiting

│   │   └── validate.js           # express-validator result che
cker

│   ├── routes/
```

```
│   │   ├── auth.js               # Signup, login, profile, avat
ar

│   │   ├── recipes.js            # CRUD + trending + remix + li
ke

│   │   ├── ai.js                 # 8 AI endpoints

│   │   ├── planner.js            # Meal plan CRUD

│   │   ├── grocery.js            # Grocery list + auto-generate

│   │   ├── admin.js              # Admin user/recipe/analytics

│   │   ├── chat.js               # DM feedback system

│   │   ├── social.js             # Follow, feed, posts, likes

│   │   ├── pantry.js             # Pantry CRUD

│   │   └── users.js              # Public user profile lookup

│   ├── services/

│   │   └── ai.js                 # Groq + Imagga integration (3
05 lines)

│   └── utils/

│       ├── recipeImageUrl.js     # Dynamic recipe image resolve
r

│       └── recipeImageHeuristic.js # Fallback image URL builder

├── frontend/

│   ├── index.html                # Login/signup page

│   ├── dashboard.html            # Main user dashboard (55KB)

│   ├── admin.html                # Admin portal

│   ├── profile.html              # Public user profile

│   ├── css/

│   │   └── styles.css            # Complete design system

│   └── js/
```

```
│       ├── auth.js               # Token management, apiFetch h
elper

│       ├── dashboard.js          # Main app logic (1315 lines)

│       ├── planner.js            # Calendar + meal plan UI

│       ├── grocery.js            # Grocery list UI + budget tra
cker

│       ├── chatbot.js            # AI chatbot UI

│       ├── chat.js               # DM messaging UI

│       ├── social.js             # Community feed UI

│       ├── pantry.js             # Pantry management UI

│       ├── voice.js              # Speech recognition + TTS

│       ├── timer.js              # Cooking timer module

│       ├── nutrition.js          # Nutrition analytics dashboar
d

│       ├── profile.js            # User profile page logic

│       ├── admin.js              # Admin portal logic (26KB)

│       ├── ai_chat.js            # AI chat initialization

│       └── recipeImageHeuristic.js # Client-side image fallback

├── database/

│   └── schema.sql                # Complete schema (536 lines)

├── package.json                  # Dependencies and scripts

├── vercel.json                   # Deployment configuration

└── .env.example                  # Environment variable templat
e
```

5.2 KEY IMPLEMENTATION PATTERNS

5.2.1 AI Response Parsing

All Groq responses are processed through the `parseAiJson()` helper which strips markdown code fences before `JSON.parse()`. This handles the common LLM behavior

of wrapping JSON in code blocks.

5.2.2 Upsert Pattern (Grocery Lists)

The grocery module implements a manual upsert: it first queries for an existing list, then either updates or inserts. This ensures each user has exactly one active grocery list.

5.2.3 Trigger-Based Counters

Like counts on recipes and posts are managed entirely by PostgreSQL triggers, ensuring atomicity and consistency regardless of which client or service makes the change.

5.2.4 SPA-Like Navigation

The dashboard uses JavaScript-based section switching (`showSection()`) rather than separate pages. Each section is a hidden

that becomes visible when its nav item is clicked. Data is lazy-loaded only when a section is activated.

5.3 DEPLOYMENT CONFIGURATION

The application supports three deployment targets:

- **Railway:** Auto-detected Node.js deployment with environment variable injection.
- **Render:** Web service with `npm install` build and `npm start` entry.
- **VPS (Ubuntu):** PM2 process manager with nginx reverse proxy.

The `vercel.json` configuration routes all requests through the Express server to support the SPA fallback pattern.

---

6. TESTING AND VALIDATION

6.1 TESTING METHODOLOGY

The project employs a combination of manual API testing (via cURL), integration testing through the live frontend, and automated health checks. Each module was tested independently before integration.

6.2 API TESTING

6.2.1 Authentication Flow Testing

```
TEST SIGNUP

curl -X POST http://localhost:3000/api/auth/signup \

  -H "Content-Type: application/json" \

   -d  '{"email":"test@test.com","password":"Test1234!","usernam
e":"chef1"}'
```

```
EXPECTED: 201 { "MESSAGE": "ACCOUNT CREATED SUCCESSFULLY", "USERID": "..." }
```

```
TEST LOGIN
```

```
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"Test1234!"}'
```

```
EXPECTED: 200 { "TOKEN": "EYJ...", "USER": { "ID": "...", "ROLE": "USER" } }
```

```
TEST INVALID LOGIN
```

```
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"wrong@test.com","password":"wrong"}'
```

```
EXPECTED: 401 { "ERROR": "INVALID EMAIL OR PASSWORD" }
```

6.2.2 AI Module Testing

```
TEST AI SUGGEST
```

```
curl -X POST http://localhost:3000/api/ai/suggest \
```

```
  -H "Authorization: Bearer YOUR_TOKEN" \

  -H "Content-Type: application/json" \

  -d '{"ingredients":["chicken","garlic","lemon","spinach"]}'


EXPECTED: 200 { "RECIPES": [...4 RE
CIPE OBJECTS...] }


TEST AI CHAT

curl -X POST http://localhost:3000/api/ai/chat \

  -H "Authorization: Bearer YOUR_TOKEN" \

  -H "Content-Type: application/json" \

  -d '{"message":"What can I make with eggs and cheese?","histor
y":[]}'


EXPECTED: 200 { "REPLY": "..." }
```

6.2.3 Health Check Testing

```
curl http://localhost:3000/api/health

EXPECTED: 200 { "STATUS": "OK", "TI
MESTAMP": "2026-04-01T..." }
```

6.3 INPUT VALIDATION TESTING

Each endpoint was tested with invalid inputs to verify the `express-validator` middleware:

Test Case	Input	Expected Response
Empty email on signup	`{"email":"","password":"Test1234!","username":"a"}`	400: "Valid

		email required"
Weak password	`{"email":"x@x.com","password":"abc","username":"test"}`	400: "Password must be at least 8 characters"
Short username	`{"email":"x@x.com","password":"Test1234!","username":"a"}`	400: "Username must be 2–30 characters"
Empty ingredients array	`{"ingredients":[]}`	400: "Provide at least one ingredient"
Invalid meal type	`{"meal_type":"Midnight"}`	400: "Invalid meal type"
Missing recipe title	`{"category":"Lunch","cooking_time":"20 mins"}`	400: "Title must be 3–100 chars"

6.4 RATE LIMITING TESTING

Rate limiters were tested by sending rapid sequential requests:

- **AI Limiter:** Confirmed 429 response after 20 requests within 15 minutes.
- **Auth Limiter:** Confirmed 429 after 10 login attempts within 15 minutes.
- **General Limiter:** Confirmed 429 after 100 recipe list requests within 15 minutes.

6.5 SECURITY TESTING

Test	Method	Result
Access protected route without token	GET `/api/auth/me` (no header)	401: "Missing or invalid Authorization header"
Access admin route as regular user	GET `/api/admin/users` with user token	403: "Forbidden: Admin access required"
Delete another user's recipe	DELETE `/api/recipes/:id` with non-owner token	403: "Not authorized"
Self-deletion prevention (admin)	DELETE `/api/admin/users/:ownId`	400: "Cannot delete your own account"
SQL injection attempt	Search with `'; DROP TABLE--`	Handled safely by parameterized Supabase queries

6.6 CROSS-BROWSER TESTING

The frontend was tested across the following browsers:

- Google Chrome 120+ ✅
- Mozilla Firefox 120+ ✅
- Microsoft Edge 120+ ✅
- Safari 17+ ✅ (Voice features limited to WebKit-based browsers)

6.7 RESPONSIVE DESIGN TESTING

The application was tested at the following breakpoints:

- **Mobile (320px–767px):** Sidebar collapses to hamburger menu. Recipe cards stack vertically. Login page shows only the auth card.
- **Tablet (768px–1023px):** Login hero section appears. Two-column recipe grid.
- **Desktop (1024px+):** Full sidebar navigation. Three-column recipe grid. Side-by-side layout for forms.

---

7. RESULTS AND DISCUSSION

7.1 SYSTEM PERFORMANCE

Metric	Value
Server startup time	< 2 seconds
API response time (database queries)	50–150ms average
AI recipe generation (Groq)	2–5 seconds for 4 recipes
AI meal plan generation (Groq)	5–10 seconds for 21 recipes
Fridge scan (Imagga)	1–3 seconds
Frontend initial load	< 1.5 seconds (static files)
Database query with RLS	< 100ms for standard CRUD

7.2 AI OUTPUT QUALITY

The Groq-powered Llama 3 70B model consistently produces:

- **Accurate JSON structure:** The `parseAiJson()` helper successfully parses 95%+ of responses without intervention.
- **Diverse recipes:** The prompt engineering ensures 4 distinct cuisines per suggestion batch.
- **Nutritionally plausible outputs:** Calorie and macro estimates are within reasonable ranges for the described dishes.
- **Smart scaling:** Non-linear adjustments for spices and leavening agents produce more realistic scaled recipes than simple multiplication.

7.3 USER INTERFACE RESULTS

The frontend achieves a premium aesthetic through:

- **Glassmorphism design:** Semi-transparent cards with blur effects and gradient borders.
- **Dark/Light theme toggle:** Complete theme switching with persistent preference in localStorage.

- **Micro-animations:** Smooth transitions on section switches, hover effects on cards, and loading spinners.
- **Responsive navigation:** Collapsible sidebar with overlay on mobile, persistent on desktop.
- **Accessibility:** Semantic HTML5 elements, proper form labels, and keyboard-navigable controls.

7.4 FEATURE COMPLETION MATRIX

Feature	Status	Notes
User Registration & Login	✅ Complete	Email validation, password strength, JWT
AI Recipe Suggestion	✅ Complete	4 multi-cuisine recipes per query
Smart Fridge Scanner	✅ Complete	Imagga-powered with 15+ food keywords
AI Chatbot	✅ Complete	Conversational with history context
7-Day Meal Planner	✅ Complete	Calendar UI + AI auto-generation
Grocery List (Manual + Auto)	✅ Complete	Auto-deduplicated from meal plans
Smart Pantry	✅ Complete	Expiry tracking + zero-waste AI
Recipe Upload	✅ Complete	With image upload to Supabase Storage
Recipe Remix	✅ Complete	Fork with original_recipe_id tracking
Ingredient Substitution	✅ Complete	AI-powered with explanations
Smart Recipe Scaling	✅ Complete	Non-linear scaling logic
Image-to-Recipe	✅ Complete	Imagga + Groq pipeline
Social Feed	✅ Complete	Post, like, follow system
Direct Messaging	✅ Complete	User-to-admin feedback threads
Voice Cooking Mode	✅ Complete	Speech recognition + TTS
Admin Portal	✅ Complete	User/recipe management + analytics
Gamification (XP/Ranks)	✅ Complete	4-tier rank system + challenges
Dark/Light Theme	✅ Complete	Persistent user preference

---

8. FUTURE SCOPE AND ENHANCEMENTS

8.1 SHORT-TERM IMPROVEMENTS

- **Real-time Notifications:** Implement Supabase Realtime subscriptions for live message notifications and social feed updates.
- **Offline Support:** Add a Service Worker for PWA capabilities, enabling offline recipe viewing.
- **Internationalization (i18n):** Multi-language support for the UI and AI prompts.
- **Image Recognition Upgrade:** Replace Imagga with a fine-tuned model specifically trained on food ingredients for higher accuracy.

8.2 MEDIUM-TERM FEATURES

- **Barcode Scanner:** Integrate a barcode scanning library to auto-add packaged items to the pantry with nutritional data.
- **Collaborative Meal Planning:** Allow households to share and co-edit meal plans.
- **Dietary Goal Tracking:** Weekly/monthly nutrition analytics with visual charts and goal-setting.
- **Recipe Video Integration:** Support embedded cooking video tutorials alongside text instructions.

8.3 LONG-TERM VISION

- **Mobile Application:** Native iOS/Android app using React Native or Flutter.
- **Smart Home Integration:** Connect with smart kitchen appliances (ovens, thermometers) via IoT protocols.
- **Machine Learning Recommendations:** Train a collaborative filtering model on user behavior to provide personalized recipe recommendations.
- **Marketplace:** Allow users to sell premium recipes or cooking courses within the platform.

---

9. CONCLUSION

The AI Kitchen Assistant successfully demonstrates the practical application of modern AI technologies in solving real-world culinary challenges. By combining Large Language Models (Groq/Llama 3) for intelligent recipe generation with Computer Vision (Imagga) for ingredient detection, the platform transforms the traditional recipe app paradigm from a passive database into an active, intelligent cooking companion.

Key achievements of the project include:

- A robust, secure backend architecture with 10 route modules, 4 middleware layers, and comprehensive Row Level Security across 14 database tables.
- Eight distinct AI-powered features, from basic recipe suggestions to advanced capabilities like reverse image-to-recipe engineering and non-linear recipe scaling.
- A premium, responsive frontend with glassmorphism design, dark/light theme support, voice-controlled cooking mode, and gamification elements.
- A complete social ecosystem with community feeds, follow systems, recipe remixing, and direct messaging.

The system is production-ready with deployment configurations for Railway, Render, and VPS environments, and is designed for extensibility through its modular architecture.

---

10. REFERENCES

- Vaswani, A., et al. (2017). "Attention Is All You Need." *Advances in Neural Information Processing Systems*, 30.
- Brown, T. B., et al. (2020). "Language Models are Few-Shot Learners." *Advances in Neural Information Processing Systems*, 33.
- Touvron, H., et al. (2023). "LLaMA: Open and Efficient Foundation Language Models." *arXiv preprint arXiv:2302.13971*.

- Meta AI. (2024). "Llama 3 Model Card." https://llama.meta.com/llama3/
- Groq, Inc. (2024). "Groq API Documentation." https://console.groq.com/docs
- Imagga Technologies. (2024). "Imagga Auto-Tagging API." https://docs.imagga.com/
- Supabase, Inc. (2024). "Supabase Documentation — Auth, Database, Storage." https://supabase.com/docs
- Express.js Foundation. (2024). "Express.js API Reference." https://expressjs.com/en/api.html
- Node.js Foundation. (2024). "Node.js Documentation." https://nodejs.org/docs/latest/api/
- Helmet.js. (2024). "Helmet.js Security Middleware." https://helmetjs.github.io/
- OWASP Foundation. (2023). "OWASP Top Ten Web Application Security Risks." https://owasp.org/www-project-top-ten/
- PostgreSQL Global Development Group. (2024). "PostgreSQL Row Level Security." https://www.postgresql.org/docs/current/ddl-rowsecurity.html
- Mozilla Developer Network. (2024). "Web Speech API." https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- Fielding, R. T. (2000). "Architectural Styles and the Design of Network-based Software Architectures." Doctoral Dissertation, University of California, Irvine.
- Nielsen, J. (1994). "Usability Engineering." Morgan Kaufmann Publishers.

APPENDIX A: ENVIRONMENT VARIABLES

Variable	Description	Source
`GROQ_API_KEY`	Groq Cloud API key for LLM inference	https://console.groq.com/keys
`IMAGGA_API_KEY`	Imagga API key for image analysis	https://imagga.com/dashboard
`IMAGGA_API_SECRET`	Imagga API secret	https://imagga.com/dashboard
`SUPABASE_URL`	Supabase project URL	Supabase Dashboard → Settings → API
`SUPABASE_ANON_KEY`	Supabase anonymous (public) key	Supabase Dashboard → Settings → API
`SUPABASE_SERVICE_KEY`	Supabase service role key (admin access)	Supabase Dashboard → Settings → API
`JWT_SECRET`	Secret for JWT token verification	Self-generated random string
`PORT`	Server listening port (default: 3000)	Custom
`NODE_ENV`	Environment mode (development/production)	Custom
`FRONTEND_URL`	CORS allowed origin	Custom

APPENDIX B: NPM DEPENDENCIES

Package	Version	Purpose
`express`	^4.18.3	Web server framework
`@supabase/supabase-js`	^2.39.7	Supabase client SDK
`@google/generative-ai`	^0.21.0	Google Generative AI SDK (legacy)
`axios`	^1.13.5	HTTP client for external API calls
`bcryptjs`	^2.4.3	Password hashing utility
`cors`	^2.8.5	CORS middleware
`dotenv`	^16.4.5	Environment variable loader
`express-rate-limit`	^7.1.5	Rate limiting middleware
`express-validator`	^7.0.1	Input validation middleware
`helmet`	^7.1.0	Security HTTP headers
`jsonwebtoken`	^9.0.2	JWT creation and verification
`multer`	^1.4.5-lts.1	File upload middleware
`uuid`	^9.0.1	UUID generation utility
`nodemon`	^3.1.0	Development auto-restart (devDep)